

The dangling else problem is a syntactic ambiguity which programming languages usually encounter when an else clause is potentially applicable to two or more preceding if statements, thus creating a situation where it is difficult to tell to which if the else belongs. Such case could be illustrated with the following pseudo-code: `if (condition1) if (condition2) statement1 else statement2` – in this case, the else could be said to belong to either the outer if (condition1) or the inner if (condition2), consequently producing two different parse trees and execution semantics. The majority of compilers, mine included, resolve this ambiguity through applying the "most recent if" or "nearest if" rule that ties each else to the closest unmatched if statement preceding it. My compiler's parser has this implemented through the grammar production rules and precedence handling: whenever the parser comes across an else keyword, it immediately links it with the most recently parsed if statement that has not yet been paired with an else, thus implicitly creating right-associative nesting. This means that in the above example, statement2 will be executed when condition1 is true but condition2 is false (bound to inner if) and not when condition1 is false. To make the situation clear that the alternative interpretation is meant, the use of braces or begin-end blocks is required from programmers' side: `if (condition1) { if (condition2) statement1 } else statement2` is now unambiguously tying the else to the outer if. This method is in line with the behavior of C, Java, and the majority of mainstream languages, which results in a consistent and predictable control flow while at the same time only requiring very little lookahead during parsing.